

BRICS Health Resilience: ML System Blueprint v1

Owner: AI/ML lead. **Scope:** everything from data ingestion to the final prediction/alert JSON.

Principle (locked): AI predicts the future. The redistribution engine makes decisions. The backend coordinates everything.

The ML system emits **one record per PHC x medicine x cycle**. It never recommends transfers.

Honesty note. Operational data here is simulated (calibrated to real WHO/World Bank indicators). Every metric in this document is an **acceptance gate (a target)**, not a result. Report real numbers only after running the experiments, and always label them "on simulated data".

0. Decisions at a glance

Component	Exact approach	Family	Why
Data pipeline	Bronze / Silver / Gold parquet layers, pandera contracts, provenance tag per field	Data engineering	Reproducible, testable, defensible (REAL / DERIVED / SIMULATED)
Preprocessing	Reconciliation, censoring flags , gap masking, per- series scaling, time-based splits	Statistical	Stockouts hide true demand; naive cleaning trains a model that under- forecasts
Forecasting	DemandNet : global PyTorch GRU + tabular MLP + medicine embedding, direct 7-day, quantile heads (P10/P50/P90) , censored pinball loss	Supervised, probabilistic time-series	One small model serves all PHCs and medicines, gives intervals (needed for safety stock and risk), and is federation- friendly
Forecast benchmarks	Seasonal naive, moving average, LightGBM (Tweedie)	Baselines	Champion must beat these on validation; not

Component	Exact approach	Family	Why
			chosen for looking impressive
Anomaly detection	Layered: robust residual z-score + CUSUM, Isolation Forest per country, district co-movement; fused into one score	Hybrid statistical + unsupervised	Residual z catches spikes fast; IF catches odd combinations; spatial layer separates outbreaks from data errors
Stockout prediction	Depletion simulation on forecast trajectory (ETA) + probability from quantile-derived window distribution	Time-series derived	Exact, explainable, coherent with the forecast
Safety-stock optimization	Quantile/newsvendor safety stock with lead-time variance, plus Lagrangian capacity allocation across medicines	Optimization	Service level tied to medicine criticality; respects storage limits
Risk scoring	Weighted composite (probability, urgency, anomaly, cover gap) x criticality, hard overrides, isotonic calibration	Hybrid rule + calibrated	Explainable on day 1, calibrated once outcomes exist
Alerts	Rule engine + hysteresis state machine on the output JSON	Rules	Deterministic, auditable, avoids alert fatigue
Federation	Flower, FedAvg (FedProx if non-IID), one client per country, optional local head fine-tune	Federated learning	Raw data never leaves the country node

1. Output contract (exact JSON)

The ML service returns exactly this structure and nothing else (`extra="forbid"` in the schema).

```
{
  "timestamp": "2026-09-21T08:00:00Z",
  "phc_id": "IN_UP_024",
  "country_code": "IN",
  "state_region": "Uttar Pradesh",
  "district": "Meerut",
  "medicine_id": "MED_001",
  "medicine_name": "Paracetamol",
  "unit": "tablet",
  "current_stock": 600,
  "forecast_demand": {"daily": 720.0, "planning_horizon_days": 7,
"total": 5040},
  "stockout": {"estimated_days": 0.83, "estimated_date": "2026-09-
22T04:00:00Z"},
  "risk": {"score": 0.90, "level": "HIGH"},
  "anomaly": {"score": 0.72, "detected": true},
  "inventory_protection": {"safety_stock": 1000, "lead_time_hours": 8}
}
```

Field derivation

Field	Source	Computation
timestamp	Cycle clock	as_of of the run, UT ISO-8601 with Z
phc_id, country_code, state_region, district	phc_master	Copied
medicine_id, medicine_name, unit	medicine_catalog	Copied
current_stock	Latest reconciled inventory	Integer units at timestamp
forecast_demand.total	DemandNet, surge-adjusted P50	Sum over the planning horizon, rounded to in
forecast_demand.daily	Derived	total / planning_horizon_1 decimal
forecast_demand.planning_horizon_days	Config	Default 7
stockout.estimated_days	Depletion simulation	No-resupply ETA in d: decimals). Capped at meaning "no stockout foreseeable". 0.0 iff stock is 0

Field	Source	Computation
<code>stockout.estimated_date</code>	Derived	<code>timestamp + unrot estimated_days</code>
<code>risk.score</code> , <code>risk.level</code>	Risk module	Score in [0,1], 2 decim Level: LOW < 0.30, MEDIUM < 0.60, HIGH 0.60
<code>anomaly.score</code> , <code>anomaly.detected</code>	Anomaly module	Score in [0,1]; detecte score >= tau or hard r fires
<code>inventory_protection.safety_stock</code>	Optimizer	Integer units, <code>ceil</code>
<code>inventory_protection.lead_time_hours</code>	Replenishment history	Median of last 10 obs replenishments, fallback: catalog default

Invariants (enforced by tests and runtime validation)

- `risk.level` matches `risk.score` thresholds (score is rounded before the level is assigned).
- `anomaly.detected == true` implies `anomaly.score >= tau`.
- `stockout.estimated_days == 0` iff `current_stock <= 0`; `estimated_date >= timestamp`.
- `abs(total - daily * horizon) <= 0.5 * horizon` (rounding tolerance).
- `safety_stock >= 0`, all scores in [0, 1], no NaN/inf.

What downstream needs (nothing else)

For Sharma's engine, everything derives from this JSON:

```
protected_stock      = forecast_demand.total +
inventory_protection.safety_stock
transferable_surplus = max(0, current_stock - protected_stock)
required_stock       = forecast_demand.total +
inventory_protection.safety_stock
deficit              = max(0, required_stock - current_stock)
```

No double counting: `safety_stock` covers demand **uncertainty** over lead time plus review period. The **expected** demand over lead time is already inside `forecast_demand.total` (horizon >= lead time).

Stored internally, deliberately not in the contract

P10/P90 quantiles, `p_stockout_window`, anomaly type and component scores, risk components, model version, feature-snapshot hash. They live in the `prediction_meta` table and the `/v1/explain` endpoint. Prediction intervals therefore stay available without changing the contract.

2. End-to-end architecture

```
flowchart TD
    subgraph SRC ["Data sources"]
        WHO["WHO GHO and World Bank<br/>REAL indicators"]
        SIM["PHC simulator<br/>SIMULATED operations + event labels"]
        LIVE["Live PHC cycle feed<br/>MVP-2"]
    end
    subgraph DP ["Data pipeline"]
        BR["Bronze: raw"] --> SL["Silver: validated"] --> GD["Gold: features and labels"]
    end
    subgraph TR ["Training plane"]
        LOC["Per-country local training"]
        FLS["Flower server: FedAvg"]
        REG["Model registry"]
    end
    subgraph INF ["Inference plane"]
        PRED["Predictor pipeline"]
        OUT["Exact JSON record"]
    end
    subgraph DOWN ["Downstream"]
        BE["Backend: FastAPI + Postgres"]
        AL["Alert engine"]
        RD["Redistribution engine"]
        FE["Frontend"]
    end
    WHO --> BR
    SIM --> BR
    LIVE --> BR
    GD --> LOC
    LOC <--> FLS
    FLS --> REG
    REG --> PRED
    SL --> PRED
    PRED --> OUT --> BE
    BE --> AL --> FE
    BE --> RD --> FE
    RD -->|"transfer applied, new stock"| LIVE
```

Steps

1. Real indicators and simulated operations land in Bronze; live cycle data joins in MVP-2.
2. Silver holds validated tables; Gold holds training-ready features and labels.
3. Each country node trains locally; Flower aggregates weights only; the registry stores the champion.
4. The predictor reads Silver (last 60 days), runs the pipeline, and emits the JSON.
5. Backend stores it, the alert engine and redistribution engine consume it, and applied transfers change stock, which feeds the next cycle.

3. Data pipeline

3.1 Layers and provenance

Layer	Content	Provenance tag
External	Population, disease burden, health-worker density (WHO GHO, World Bank)	REAL
Simulated	PHC footfall, consumption, stock, deliveries, staff, beds	SIMULATED
Derived	Stock cover, transferable surplus, stockout days	DERIVED
Model output	Forecast, anomaly score, risk	ML OUTPUT

Every column in the data dictionary carries a `provenance` attribute. Storage is Parquet partitioned by `country_code/date`, queried with DuckDB or pandas. Postgres is used only for serving (predictions, alerts, meta).

```

flowchart TD
    A["WHO GHO API, World Bank<br/>REAL"] --> B1
    S["PHC simulator, calibrated to A<br/>SIMULATED + labels table"] --> B1
    B1
    L["Live cycle feed, MVP-2"] --> B1
    B1["Bronze: immutable raw parquet"] --> V1["Validate: pandera schemas"]
    V1 -->|"fail"| Q["Quarantine + data-quality report"]
    V1 -->|"pass"| C1["Clean: dedup, UTC, unit normalize"]
    C1 --> C2["Reconcile stock<br/>open + received - dispensed = close"]
    C2 --> C3["Flag censoring and gaps"]
    C3 --> SV["Silver: phc_master, inventory, demand_history"]
    SV --> F1["Feature build as-of t"]
  
```

```

F1 --> LB["Label build: y t+1..t+7, stockout flags"]
LB --> SP["Time split + cold-start PHC holdout"]
SP --> GD["Gold: training parquet + feature_spec.json"]

```

Steps: validate against contracts, quarantine failures, clean, reconcile stock, flag censored/missing days, publish Silver, then build features and labels as-of each timestamp, split by time, and publish Gold.

3.2 Tables (extends blueprint sections 9 to 11)

Table	Key columns	Additions to blueprint
phc_master	phc_id , country, region, district, area_type, population_served, lat/lon	lat , lon , timezone
inventory_snapshot	phc_id , medicine_id , timestamp , opening/received/dispensed/current stock, reorder_level, safety_stock (policy), lead_time_hours	report_delay_h , is_censored
demand_history	date , phc_id , medicine_id , footfall, consumption, stock start/end, staff, beds	is_censored , imputed , recon_error
medicine_catalog	medicine_id , name, unit, category	criticality_class (CRITICAL / ESSENTIAL / STANDARD), shelf_life_days , volume_per_unit , default_lead_time_h
replenishments	phc_id , medicine_id , ordered_at, received_at, quantity	Needed for lead-time mean and sigma
event_labels (simulation only)	event_id , type, phc/district, start, end, magnitude	Ground truth for anomaly evaluation; never a model feature

3.3 Simulator requirements (Anant builds; these are ML-critical)

1. **Demand:** latent rate = population_served x incidence(season, disease signal) x visit rate x prescription rate. Footfall and consumption drawn from negative binomial (overdispersed counts) with day-of-week and holiday effects.
2. **Supply:** (s, S) replenishment with lognormal lead times and occasional late deliveries.

3. **Censoring:** `dispensed = min(demand, available)`. Store the observed value as consumption and keep `true_demand` in a separate evaluation-only table.
4. **Events:** outbreak (ramp then decay), festival surge, supply disruption, data glitches (unit x10 error, duplicate, missing day). All logged in `event_labels`.
5. **Heterogeneity across countries** (scale, seasonality, missingness). This makes federation meaningful because data is non-IID.
6. **Independence guard against circularity:** the generator must not share structure with the model (no GRU-like dynamics), test scenarios must be unseen families, and the forecasting pipeline should also be sanity-checked on **one real public pharmacy-sales dataset** (verify licence) so results are not purely simulator self-agreement.

Recommended size: at least 2 years, 100+ PHCs per country, 15 medicines.

4. Preprocessing

#	Step	Rule	Reason
1	Typing and units	UTC timestamps; convert to base unit from catalog	Consistency
2	Dedup	Keep latest per (phc, medicine, timestamp)	Late corrections
3	Reconciliation	Flag if <code>abs(open + recv - disp - close) > max(2% of open, 1 unit)</code>	Catch bad reports; use <code>close</code> as truth for <code>current_stock</code> , exclude the row from training targets
4	Censoring flag	<code>is_censored = close <= 0 or disp >= open + recv - eps</code>	Observed consumption is only a lower bound on demand
5	Gaps	Reindex to full calendar. Gap <= 2 days: linear interpolation for features, <code>imputed=1</code> . Longer: masked out of targets and windows with more than 20% missing	Avoid fabricating history

#	Step	Rule	Reason
6	Target outliers	Winsorize training targets only at rolling median +/- 6 MAD, and only outside known or detected events	Stability. The anomaly detector reads unwinsorized data
7	Scaling	s_i = mean uncensored consumption of series i over the trailing training window; model works on y / s_i	Scale-free, so a global and federated model transfers across PHCs and countries. Scalers stay local to the country node
8	Splits	Rolling-origin backtest: train to T0, purge 7 days, validate next 28 days, test the final 56 days (contains held-out events). 3 expanding folds. Plus cold-start holdout : 10% of PHCs never seen in training	No leakage from overlapping 7-day targets; tests new-PHC generalization

5. Feature engineering

```

flowchart TD
    SV["Silver tables"] --> AS["As-of filter: rows with ts ≤ t only"]
    AS --> G1["Demand history<br/>lags, rolling, momentum"]
    AS --> G2["Calendar and holidays"]
    AS --> G3["Operational<br/>footfall, staff, stock cover"]
    AS --> G4["Cross-sectional<br/>district peers, Phase 2"]
    ST["Static: catalog, PHC master, WHO priors"] --> G5["Static + embeddings"]
    EX["Exogenous: disease signal, weather, Phase 2"] --> G6["Exogenous"]
    G1 --> SC["Per-series scaling"]
    G3 --> SC
    SC --> ASM["Assemble: sequence tensor 28 x d + tabular vector"]
    G2 --> ASM
    G4 --> ASM
    G5 --> ASM
    G6 --> ASM
    ASM --> PAR["Parity test: offline features equal online features"]

```

Steps: filter to data available at time t, compute each feature group, scale, assemble the GRU sequence tensor and the tabular vector, then verify offline/online parity.

Group	Features	Phase
A. Demand history	lag 1,2,3,7,14,28 (scaled); rolling mean 3/7/14/28; rolling std 7/14; EWM(0.3); 7-day slope; momentum = mean3 / mean28 ; same-weekday mean of last 4 weeks; days since last censored day; censored fraction (14d)	1
B. Calendar	day-of-week sin/cos, month sin/cos, day-of-year sin/cos, is_holiday, days to holiday (per-country holidays), hemisphere-aware season	1
C. Operational	footfall lag 1 and rolling 7; consumption per visit (rolling 7); footfall momentum; doctors_present / required; nurses ratio; stock cover days = stock / rolling mean 7; days since delivery; last lead time	1 (beds in 2)
D. Static	medicine embedding (dim 8), criticality class, area_type, log population_served	1
E. Cross-sectional	district peer momentum (same medicine, excluding self); district footfall z; share of peers alerting	2
F. Exogenous	disease signal index (fever / diarrhoea / respiratory proxy), temperature, rainfall; WHO priors (disease burden, health-worker density, tagged REAL)	2

Leakage rules

1. Every feature uses rows with `ts <= as_of` only.
2. `true_demand`, `event_labels`, and any label are never in the feature set.
3. Scalers are fit on the training window only.
4. A parity test compares Gold features with online-computed features for 100 random keys (tolerance 1e-6). One `features.py` serves both training and inference.

6. Demand forecasting

6.1 Model ladder

Rank	Model	Role
1	Seasonal naive (same weekday last week), 7-day moving average	Baselines; the floor to beat


```

    loss = torch.maximum(q * err, (q - 1) * err)      # standard
pinball
    ignore = cens.unsqueeze(-1) & (err < 0)          # demand >= y, so
over-prediction is fine
    loss = torch.where(ignore, torch.zeros_like(loss), loss)
    m = valid.unsqueeze(-1).float()
    return (loss * m).sum() / m.sum().clamp_min(1.0) / len(qs)

```

6.4 Training settings

Item	Value
Window / horizon	L = 28 days in, H = 7 days out
Optimizer	AdamW, lr 2e-3, weight decay 1e-4, cosine schedule, grad clip 1.0
Batch / epochs	512, max 40, early stopping patience 6 on validation WAPE
Seeds	3, report mean and spread
Data loading	Lazy sliding windows from Gold parquet (never materialize all windows)
Interval calibration	Conformal correction (CQR) of P10/P90 per horizon on the latest validation fold, target 80% coverage

6.5 Post-processing and cold start

- Clip to ≥ 0 ; cap at 5x the series historical max; enforce quantile order.
- **Cold start (< 28 days history)**: use the district-level per-capita rate for that medicine x population served, widen intervals by 1.5x, and log `model_used = cold_start` in meta.
- Intermittent items (for example syringes or PPE with many zero days): the same model applies, but the safety stock uses empirical residual quantiles instead of a normal assumption.

7. Federated learning

```

sequenceDiagram
    participant S as Flower server
    participant IN as India node
    participant BR as Brazil node
    participant ZA as South Africa node
    S->>IN: global weights, round r

```

```

S->>BR: global weights, round r
S->>ZA: global weights, round r
Note over IN,ZA: local epochs on local data only
IN-->>S: weights and n_samples
BR-->>S: weights and n_samples
ZA-->>S: weights and n_samples
S->>S: aggregate weighted by n_samples
S->>S: log per-country validation WAPE

```

Item	Design
Clients	One per country (extendable to Russia and China). Each holds many PHCs and one local dataset
Algorithm	FedAvg weighted by samples. FedProx ($\mu = 0.01$) if non-IID drift hurts
Rounds	20 to 30, 2 to 3 local epochs per round
Shared	Model weights only. Raw data, scalars, and the anomaly models stay local
Personalization	Optional: 1 to 2 local epochs fine-tuning the head after the last round
Not federated	Isolation Forest (trees do not average) is trained per country. Its scores are converted to percentile ranks so they are comparable across countries. Risk and safety-stock modules are deterministic
Privacy extras (future)	Secure aggregation, DP-SGD (Opacus). Do not claim formal privacy without them

Experiment (the headline slide): Local-only vs Centralized vs Federated (vs Federated + personalized). Report WAPE per country **and worst-country WAPE**, plus rounds to converge. Also test transfer to an unseen country node.

Client skeleton (verify API against the pinned Flower version):

```

class CountryClient(fl.client.NumPyClient):
    def get_parameters(self, config):
        return [v.cpu().numpy() for v in
                self.model.state_dict().values()]

    def fit(self, parameters, config):
        set_params(self.model, parameters)
        train(self.model, self.train_loader,
              epochs=config["local_epochs"],
              prox_mu=config.get("mu", 0.0), global_params=parameters)
        return self.get_parameters({}), len(self.train_loader.dataset),

```

```

{}

def evaluate(self, parameters, config):
    set_params(self.model, parameters)
    m = evaluate_wape_and_pinball(self.model, self.val_loader)
    return m["pinball"], len(self.val_loader.dataset), {"wape":
m["wape"]}

```

8. Anomaly detection

Purpose: detect abnormal **demand velocity** (spikes, sustained drift) early, and separate real surges from data errors.

```

flowchart TD
    O["Observed consumption + footfall<br/>daily or 5h cycle"] --> R["Residual vs forecast<br/>z = obs - P50 over sigma_day"]
    R --> F["Forecast P10, P50, P90"] --> R
    R --> S1["Layer 1: robust z + CUSUM"]
    O --> FE2["Multivariate feature vector"]
    FE2 --> S2["Layer 2: Isolation Forest per country"]
    PE["District peers z"] --> S3["Layer 3: spatial co-movement"]
    S1 --> FU["Fuse: 0.5 s_stat + 0.3 s_if + 0.2 s_spatial"]
    S2 --> FU
    S3 --> FU
    FU --> TH["score ≥ tau or z ≥ 5,<br/>with persistence"]
    TH -->|"no"| N["detected = false"]
    TH -->|"yes"| TY["Classify type"]
    TY -->|"SPIKE or DRIFT"| SG["Surge overlay on forecast"]
    TY -->|"DATA_QUALITY"| DQ["Route to data-quality alert, no overlay"]

```

Layers

Layer	Method	Detail
1. Statistical	Forecast-residual z	$\text{sigma_day} = (P90 - P10) / 2.563, \text{ floored at } \max(0.05 * P50, 1)$ $z = (\text{obs} - P50) / \text{sigma_day}$ $s_stat = \text{clip}((z - 2) / 4, 0, 1)$ $\text{CUSUM} (k = 0.5, h = 4) \text{ on } z \text{ flags sustained drift}$

Layer	Method	Detail
2. Unsupervised	Isolation Forest (200 trees) per country	Features: z, footfall z, consumption-per-visit ratio vs 28d, stock drawdown rate, momentum, peer share, staff ratio, intraday velocity. s_{if} = ECDF percentile of the anomaly score against the country's training distribution. Retrain weekly on 90 days
3. Spatial	District co-movement	$s_{spatial}$ = share of same-district PHCs with $z > 2$. If a PHC has no peers, renormalize the weights over layers 1 and 2

Decision rule and typing

- **Detected** if $score \geq \tau$ (default 0.70, tuned on validation) with persistence of 2 consecutive daily observations, **or immediately** if $z \geq 5$. When the hard rule fires, $score = \max(score, \tau)$ so the invariant holds.
- **Type (internal meta):**

Type	Condition	Effect
SPIKE	$z \geq 3$ and (footfall $z \geq 2$ or peer share ≥ 0.3)	Surge overlay
DRIFT	CUSUM alarm	Surge overlay (damped)
DATA_QUALITY	consumption jump without footfall or peers, reconciliation error, x10 unit jump	No overlay; data-quality alert
DROP	$z \leq -3$ with censoring or stockout	Not flagged as anomaly; handled as censored demand

Surge overlay (the "recalculate forecast" step in MVP-2)

Velocity ratio $v = \text{observed cumulative consumption so far} / (f(\text{hour}) * \text{daily forecast})$ where $f(\text{hour})$ is the historical intraday share (default: linear over clinic hours). Multiplier $m = \text{clip}(v, 1, 4)$, applied as $m(d) = 1 + (m - 1) * 0.5^{(d / 3)}$ for day d, with half-life 3 days (tune on simulated events). The adjusted forecast feeds ETA, risk, and safety stock. The safety-stock sigma is also inflated by $(1 + 0.5 * \text{anomaly_score})$.

9. Stockout prediction

```
flowchart TD
    A["Adjusted forecast P50 per day"] --> B["Extend to 30 days  
using<br/>3-day run-rate"]
    S["current_stock"] --> C["Cumulative demand curve"]
    B --> C
    C --> D["First day where cumulative ≥ stock"]
    D --> E["Linear interpolation inside the day<br/>estimated_days"]
    E --> F["estimated_date = timestamp + days"]
    Q["P10 and P90 quantiles"] --> G["Window demand distribution<br/>mu  
and sigma"]
    G --> H["P(stockout within window W)"]
    H --> RK["To risk score"]
    E --> RK
```

Steps: build the cumulative demand curve from the adjusted forecast, find when it crosses current stock, interpolate inside the day, and convert to a date. In parallel, turn the quantile forecast into a demand distribution over the risk window to get a probability.

The ETA is a **no-resupply** ETA: it answers "when does this PHC run out if nothing arrives?". Risk then compares that against lead time. In-transit orders can be included later as an optional input.

```
import numpy as np
from scipy.stats import norm

def stockout_eta(stock, fc_daily, cap_days=30):
    if stock <= 0:
        return 0.0
    run_rate = float(np.mean(fc_daily[-3:]))
    fc = np.concatenate([fc_daily, np.full(max(0, cap_days -
len(fc_daily)), run_rate)])
    cum = np.cumsum(fc)
    idx = int(np.searchsorted(cum, stock))          # first day with
cum >= stock
    if idx >= len(cum):
        return float(cap_days)
    prev = cum[idx - 1] if idx > 0 else 0.0
    return float(idx + (stock - prev) / max(fc[idx], 1e-9))

def window_demand(q, window_days, rho=0.25):
    """q: [H,3] adjusted P10/P50/P90 per day. Returns (mu, sigma) of
cumulative demand."""
    n = int(np.ceil(window_days))
    if n > len(q):
        q = np.vstack([q, np.repeat(q[-3:].mean(0, keepdims=True), n -
len(q), 0)])
```

```

w = np.clip(window_days - np.arange(len(q)), 0, 1) # fraction of
each day inside the window
mu = float((q[:, 1] * w).sum())
s = (q[:, 2] - q[:, 0]) / 2.563 * w # daily sigma
from the 80% interval
var = (1 - rho) * (s ** 2).sum() + rho * s.sum() ** 2 # pairwise
day correlation rho
return mu, float(np.sqrt(var))

def p_stockout(stock, q, window_days, rho=0.25):
mu, sig = window_demand(q, window_days, rho)
return float(1 - norm.cdf((stock - mu) / max(sig, 1e-9)))

```

Risk window: $W = \text{lead_time_days} + \text{review_period_days} + \text{decision_buffer_days}$
(buffer default 1 day for detection, approval, and dispatch).

Challenger: a calibrated LightGBM classifier for "stockout within 72h" is trained only for **evaluation** (does the forecast-derived probability match a directly supervised one?). It is not in the serving path.

10. Safety-stock optimization

10.1 Formulation

Safety stock protects against demand uncertainty over the exposure period $T = L + R$ (lead time plus review period; $R = 1$ day for daily cycles, 5/24 day for 5-hour cycles).

```

mu_T, sigma_T    from window_demand(q, T)                (section
9)
var_total        = sigma_T^2 + (mu_d * sigma_L)^2         (adds
lead-time variability)
SS(alpha)        = z(alpha) * sqrt(var_total)

```

- σ_L is the std of the last 10 observed lead times in days, floored at 10% of the mean.
- Service level from a **newsvendor critical ratio**: $\alpha^* = C_u / (C_u + C_o)$, then clipped to $[0.90, 0.995]$. C_u is a health-harm-weighted shortage cost and C_o the holding plus expiry cost, both set per criticality class in config.
- Capacity constraint:** storage volume C . With multiplier $\lambda \geq 0$:
 $\alpha_m(\lambda) = C_{u,m} / (C_{u,m} + C_{o,m} + \lambda * v_m)$. Find the smallest λ with $\sum(v_m * SS_m) \leq C$ by bisection. This is a closed-form Lagrangian solution, so no solver is needed (OR-Tools remains optional later).

Class	Examples	Service level floor	Cu : Co	Risk multiplier kappa
CRITICAL	Insulin, oxygen, IV fluids, antimalarial (endemic areas)	0.99	99 : 1	1.15
ESSENTIAL	Amoxicillin, azithromycin, salbutamol, antihypertensive, antidiabetic, ORS	0.975	39 : 1	1.05
STANDARD	Paracetamol, analgesic, antiseptic, syringes, PPE	0.95	19 : 1	1.00

Class assignments are a config decision to be confirmed against an essential-medicines list. Do not present them as clinical facts.

```

flowchart TD
    A["Quantile forecast + surge overlay"] --> B["Window L + R demand: mu, sigma with rho"]
    B --> C["Lead time mean and sigma_L"]
    C --> D["Variance including lead-time term"]
    D --> E["Service level alpha = Cu / (Cu + Co + lambda v)"]
    E --> F["SS = z(alpha) x sigma"]
    F --> G["Sum of v x SS ≤ C ?"]
    G --> H["Raise lambda by bisection"]
    H --> I["Smooth ±20 percent, ceil, minimum floor"]
    I --> OUT["inventory_protection.safety_stock"]

```

Steps: compute the demand distribution over lead time plus review, add lead-time variance, choose the service level from the medicine's cost ratio, convert to units with $z \times \sigma$, shrink service levels through λ if storage is exceeded, then smooth and round.

```

def safety_stock(sigma_LR, mu_d, sigma_L_days, alpha):
    var = sigma_LR ** 2 + (mu_d * sigma_L_days) ** 2
    return max(0.0, norm.ppf(alpha)) * np.sqrt(var)

def allocate_safety_stock(items, capacity):
    """items: dicts with Cu, Co, v (volume/unit), sigma_LR, mu_d,
    sigma_L. Bisection on lambda."""
    def ss_at(lam):
        out = []

```

```

for it in items:
    a = it["Cu"] / (it["Cu"] + it["Co"] + lam * it["v"])
    a = min(max(a, 0.90), 0.995)
    out.append(safety_stock(it["sigma_LR"], it["mu_d"],
it["sigma_L"], a))
    return np.array(out)
vol = lambda ss: float(sum(it["v"] * s for it, s in zip(items, ss)))
if vol(ss_at(0.0)) <= capacity:
    return ss_at(0.0)
lo, hi = 0.0, 1.0
while vol(ss_at(hi)) > capacity and hi < 1e6:
    hi *= 2
for _ in range(40):
    mid = (lo + hi) / 2
    lo, hi = (mid, hi) if vol(ss_at(mid)) > capacity else (lo, mid)
return ss_at(hi)

```

The alpha clip means capacity can only be met down to the 0.90 floor. If it is still exceeded, raise a `CAPACITY_INFEASIBLE` warning rather than silently under-protecting.

Stability: recomputed each cycle, then `SS_t = clip(SS_t, 0.8 * SS_{t-1}, 1.2 * SS_{t-1})`, except upward moves are uncapped when `anomaly.detected`.

11. Risk scoring

```

p_window = P(stockout within W) (section 9)
urgency = clip(1 - ETA_days / planning_horizon, 0, 1)
cover_gap = clip(1 - current_stock / reorder_level, 0, 1)
base = 0.45*p_window + 0.35*urgency + 0.10*anomaly_score +
0.10*cover_gap
score = min(1, base * kappa(criticality))
overrides : stock <= 0 -> score = 1.00
            ETA_days <= lead_days -> score = max(score, 0.75)
level : LOW < 0.30 <= MEDIUM < 0.60 <= HIGH (assigned after
rounding to 2 decimals)

```

flowchart TD

```

I1["p_window"] --> WS["Weighted sum<br/>0.45 / 0.35 / 0.10 / 0.10"]
I2["urgency = 1 - ETA / 7"] --> WS
I3["anomaly score"] --> WS
I4["cover gap vs reorder level"] --> WS
WS --> K["Multiply by criticality kappa"]
K --> OV{"Overrides"}
OV -->|"stock ≤ 0"| R1["score 1.00"]
OV -->|"ETA ≤ lead time"| R2["score at least 0.75"]
OV -->|"none"| R3["score as computed"]
R1 --> LV["Level: LOW, MEDIUM, HIGH<br/>by thresholds"]
R2 --> LV
R3 --> LV

```

```
R2 --> LV
R3 --> LV
```

```
W = dict(p=0.45, urg=0.35, anom=0.10, cover=0.10)
KAPPA = {"CRITICAL": 1.15, "ESSENTIAL": 1.05, "STANDARD": 1.0}

def risk(stock, eta_days, p_window, anomaly_score, reorder_level,
        lead_days, crit, horizon=7):
    if stock <= 0:
        return 1.0, "HIGH"
    urg = float(np.clip(1 - eta_days / horizon, 0, 1))
    gap = float(np.clip(1 - stock / max(reorder_level, 1), 0, 1))
    s = W["p"] * p_window + W["urg"] * urg + W["anom"] * anomaly_score +
    W["cover"] * gap
    s = min(1.0, s * KAPPA[crit])
    if eta_days <= lead_days:
        s = max(s, 0.75)
    s = round(float(s), 2)
    return s, ("HIGH" if s >= 0.60 else "MEDIUM" if s >= 0.30 else
    "LOW")
```

Worked check (blueprint example): stock 600, forecast 720/day gives ETA 0.83 days, urgency 0.88, `p_window` about 0.99 ($W = 8h + 1d + 1d = 2.33$ days, expected demand about 1,680), anomaly 0.72, cover gap 0.76 against reorder level 2,500. Base = $0.4455 + 0.308 + 0.072 + 0.076 = 0.90$, STANDARD class, so score 0.90, HIGH.

Tuning and calibration

1. Weights start as expert priors. Tune with a grid or constrained logistic regression (weights ≥ 0 , sum to 1) to maximize PR-AUC for "stockout within W" on simulated outcomes.
2. **Isotonic calibration** (monotone, so ordering never changes) maps the score to observed stockout frequency. Check Brier score and reliability curve. Recheck the 0.30 and 0.60 thresholds afterwards.
3. Compare against the LightGBM challenger (section 9) on PR-AUC.

12. Alert generation

Alerts are **classified from the exact JSON** (plus meta for anomaly type). The alert record persisted by the backend wraps the unchanged payload: `{alert_id, alert_type, priority, status, payload: <exact JSON>}`. That envelope is a backend concern and is **not** part of the ML contract.

Alert type	Trigger (on JSON fields)	Priority
STOCKOUT_IMMINENT	<code>current_stock <= 0</code> or <code>estimated_days <= lead_time_hours / 24</code>	P1
STOCKOUT_PROJECTED	<code>estimated_days <= planning_horizon_days</code> and <code>risk.level</code> in MEDIUM, HIGH	P2 (P1 if HIGH and ETA <= 2 days)
BELOW_SAFETY_STOCK	<code>current_stock < safety_stock</code>	P2
DEMAND_SPIKE	<code>anomaly.detected</code> and meta type SPIKE or DRIFT	P2 (P1 if risk HIGH)
DATA_QUALITY	meta type DATA_QUALITY, or no report for more than 36 h	P3
SURPLUS_AVAILABLE	<code>risk.level == LOW</code> and <code>current_stock > 1.5 * (total + safety_stock)</code>	INFO, forwarded to the redistribution engine

Ranking: `priority_score = risk.score * kappa * log10(population_served)`.

Dedup and hysteresis: key = (phc, medicine, type). Re-notify only on escalation or after 12 h. Auto-resolve after 2 consecutive LOW cycles. Rate-limit per PHC.

```
stateDiagram-v2
    [*] --> NORMAL
    NORMAL --> WATCH: risk MEDIUM or anomaly detected
    NORMAL --> ALERT: risk HIGH
    WATCH --> ALERT: risk HIGH or ETA within lead time
    WATCH --> NORMAL: risk LOW for 2 cycles
    ALERT --> ESCALATED: unresolved 6h and risk not falling
    ALERT --> MITIGATING: transfer or order approved
    ESCALATED --> MITIGATING: transfer or order approved
    MITIGATING --> WATCH: risk falls to MEDIUM
    MITIGATING --> NORMAL: risk LOW for 2 cycles
```

13. Training pipeline

```
flowchart TD
    G["Gold datasets per country"] --> BL["1. Baselines"]
    G --> GB["2. LightGBM benchmark"]
    G --> DN["3. DemandNet centralized"]
    G --> LO["4. DemandNet local-only"]
```

```

G --> FL["5. DemandNet federated"]
BL --> CMP["Compare on rolling-origin backtest"]
GB --> CMP
DN --> CMP
LO --> CMP
FL --> CMP
CMP --> SEL["Select champion by rule"]
SEL --> CQ["Conformal calibration of P10 and P90"]
CQ --> AD["Fit anomaly layer per country + tau"]
AD --> RC["Fit risk weights + isotonic calibration"]
RC --> SSP["Estimate rho, sigma_L, class costs"]
SSP --> EV["Full evaluation + closed-loop simulation"]
EV --> GATE{"Gates passed?"}
GATE -->|"yes"| REG["Register manifest + artifacts"]
GATE -->|"no"| ITER["Iterate features and hyperparameters"]
ITER --> G

```

Steps: train all five variants on identical splits, compare, pick the champion by the rule in section 6.1, calibrate intervals, fit the anomaly, risk, and safety-stock parameters, evaluate end-to-end, and register only if the gates pass.

```

python -m ml.training.train --config ml/configs/forecaster.yaml --mode
central
python -m ml.training.train --config ml/configs/forecaster.yaml --mode
local --country IN
python -m ml.federated.server --config ml/configs/federated.yaml      #
+ 3 clients
python -m ml.training.calibrate --all
python -m ml.training.evaluate --report docs/eval_report.md

```

Artifact bundle (`ml/registry/artifacts/<version>/`): `model.pt`,
`feature_spec.json` (order, versions), `scalers/<country>.json`,
`iforest_<country>.joblib`, `thresholds.yaml` (tau, risk cutoffs), `ss_params.yaml`,
`manifest.json` (git SHA, data hash, seeds, metrics).

14. Inference pipeline

```

sequenceDiagram
    participant SCH as Scheduler or Backend
    participant ML as ML service
    participant DB as Postgres and Silver store
    participant REG as Model registry
    SCH->>ML: POST /v1/predict with as_of and series
    ML->>REG: load champion artifacts, cached
    ML->>DB: read last 60 days per series up to as_of
    ML->>ML: validate then build features

```

```

ML->>ML: forecast P10 P50 P90 for 7 days
ML->>ML: anomaly score then surge overlay
ML->>ML: stockout ETA and window distribution
ML->>ML: safety stock optimization
ML->>ML: risk score and level
ML->>ML: assemble and schema-validate JSON
ML->>DB: persist records and meta
ML-->>SCH: JSON array of records
SCH->>ML: POST /v1/alerts/evaluate
ML-->>SCH: alert candidates

```

Order matters: forecast, then anomaly (needs residuals), then surge overlay, then ETA and distribution, then safety stock (needs sigma and anomaly), then risk.

```

class Predictor:
    def run(self, as_of, keys):
        obs = self.store.load_window(keys, as_of, days=60) #
        ts <= as_of only
        X = build_features(obs, self.feature_spec, as_of) #
        same code as training
        q = self.forecaster.predict(X) #
        [N,7,3] units/day
        an = self.anomaly.score(obs, q) #
        score, detected, type, velocity
        q = apply_surge(q, an)
        eta = stockout_eta(stock, q[...], 1)
        p_w = p_stockout(stock, q, window_days)
        ss = allocate_safety_stock(items, capacity)
        score, level = risk(stock, eta, p_w, an.score, reorder,
        lead_days, crit)
        return [assemble_record(...)] #
        pydantic, extra="forbid"

```

Modes and MVP-2 (intraday)

Mode	Trigger	Behaviour
Batch	Scheduler: daily (MVP-1), then 08:00 / 13:00 / 18:00 / 23:00 (MVP-2)	All series for the given countries
Event-driven	POST /v1/observations	Recompute only the touched series (plus district peers for the spatial layer)
What-if	POST /v1/whatif	Recompute one record with a hypothetical stock change; not persisted

For intraday cycles, the planning window is a rolling 7 x 24 h from `timestamp`. Day-d demand blends adjacent calendar-day forecasts by the fraction of the day elapsed, the last day reuses the last-day run-rate, and the surge overlay handles intraday velocity.

Closed loop for the demo

```
flowchart LR
    A["T0: predict<br/>risk HIGH"] --> B["Redistribution proposes transfer"]
    B --> C["what-if: recompute risk<br/>with stock + transfer"]
    C --> D["Transfer applied"]
    D --> E["Next cycle observed"]
    E --> F["Predict again<br/>MEDIUM then LOW"]
    F --> A
```

Mock predictor (unblocks the team on day 1)

`ml/inference/mock_predictor.py` returns valid, deterministic records in the exact contract from a seed and a few knobs (stock, demand, class), and is used by backend and redistribution tests before the real model exists.

15. Repository structure

Keeps the locked layout from the master blueprint. **Additions** are marked `(+)` with the reason in section 0 or in the relevant section.

```
brics-health-resilience/
├─ data/
│   ├── raw/ external/ processed/ simulated/
│   └─ schemas/                # pandera contracts,
data_dictionary source
├─ ml/
│   └─ configs/                # (+) all tunables, no magic
numbers in code
│   │   ├── data.yaml features.yaml forecaster.yaml anomaly.yaml
│   │   ├── risk.yaml safety_stock.yaml alerts.yaml federated.yaml
│   │   └─ medicine_catalog.csv # criticality class, unit, volume,
shelf life
│   │   ├── schemas/          # (+)
│   │   │   ├── inputs.py     # pydantic request models
│   │   │   └─ output.py      # THE exact JSON contract,
extra="forbid"
│   │   └─ preprocessing/
│   │       ├── clean.py transform.py features.py
│   │       └─ validation.py   # (+) reconciliation, censoring,
gaps
```

```

|   |   | models/
|   |   |   | demand_forecaster.py    # DemandNet, LightGBM benchmark
|   |   |   | baselines.py            # (+)
|   |   |   | anomaly_detector.py
|   |   |   | stockout.py              # (+) ETA and window distribution
|   |   |   | safety_stock.py          # (+) optimizer
|   |   |   | risk.py
|   |   | training/
|   |   |   | config.py  train.py  evaluate.py
|   |   |   | splits.py  backtest.py  calibrate.py  # (+)
|   |   | federated/
|   |   |   | client.py  server.py  strategy.py
|   |   |   | simulate.py          # (+) local-only vs central vs
federated runs
|   |   | inference/
|   |   |   | predictor.py            # orchestrates the pipeline
|   |   |   | assembler.py           # (+) builds and validates the JSON
|   |   |   | alerting.py            # (+) pure rule functions, called
by backend
|   |   |   | whatif.py               # (+)
|   |   |   | mock_predictor.py       # (+)
|   |   |   | service.py              # (+) FastAPI app for the ML
service
|   |   | registry/                   # (+) artifacts/<version>/,
registry.py
|   |   | monitoring/                 # (+) data_quality.py, drift.py,
performance.py, metrics.py
|   |   |   | Dockerfile              # (+)
|   |   | redistribution/  backend/  frontend/
|   |   | tests/ml/              # unit, schema, parity, invariants,
golden files
|   |   | docs/ (architecture.md, api_contract.md, ml_architecture.md = this
file, eval_report.md)

```

Key pins for `requirements.txt`: torch, flwr, lightgbm, scikit-learn, statsmodels, pandas, numpy, scipy, pyarrow, duckdb, pandera, pydantic v2, fastapi, uvicorn, holidays, mlflow, evidently, prometheus-client, pytest, ruff, mypy.

Config example (`ml/configs/risk.yaml`):

```

risk:
  weights: {p_window: 0.45, urgency: 0.35, anomaly: 0.10, cover_gap:
0.10}
  kappa: {CRITICAL: 1.15, ESSENTIAL: 1.05, STANDARD: 1.0}
  thresholds: {medium: 0.30, high: 0.60}
  overrides: {eta_le_lead_floor: 0.75}
  window: {review_period_days: 1.0, decision_buffer_days: 1.0}
  stockout: {cap_days: 30, rho: 0.25}

```

16. Evaluation

Component	Metrics	Protocol	Acceptance gate (target)
Forecast (point)	WAPE (primary; MAPE breaks on zeros), MAE, RMSE, bias, MASE	Rolling-origin backtest, by horizon, medicine, country	Beats seasonal naive by at least 15% WAPE; bias within +/-5%
Forecast (interval)	Pinball loss, P10-P90 coverage	Same	Coverage between 75% and 85%
Cold start	WAPE on held-out PHCs	Cold-start holdout	Within 1.3x of seen-PHC WAPE
Anomaly	Event-level precision / recall / F1, detection delay , false alarms per PHC-month	Against <code>event_labels</code> on unseen scenario families	Recall ≥ 0.85 , ≤ 0.5 false alarms per PHC-month, spikes $\geq 3x$ caught within 1 cycle (5h)
Stockout	Precision / recall / F1 at 24h, 72h, 7d; ETA MAE on true stockouts; PR-AUC, Brier	Against simulated actual stockouts	Recall ≥ 0.85 at precision ≥ 0.60 (72h); median warning lead time $\geq$ lead_time_hours
Risk	ECE / reliability curve, AUROC, PR-AUC vs challenger	Held-out test	Calibrated within 0.05 ECE; PR-AUC not worse than challenger by more than 0.02
Safety stock	Realized cycle service level vs target alpha, fill rate, excess inventory days	Closed-loop simulation	Service level within +/-2 points of target; less excess than a fixed 7-day rule at equal service
Alerts	Alert precision, recall of stockouts pre-warned, alerts per PHC-week, time-to-alert	Closed-loop simulation	Precision ≥ 0.60 ; alert volume bounded (no fatigue)

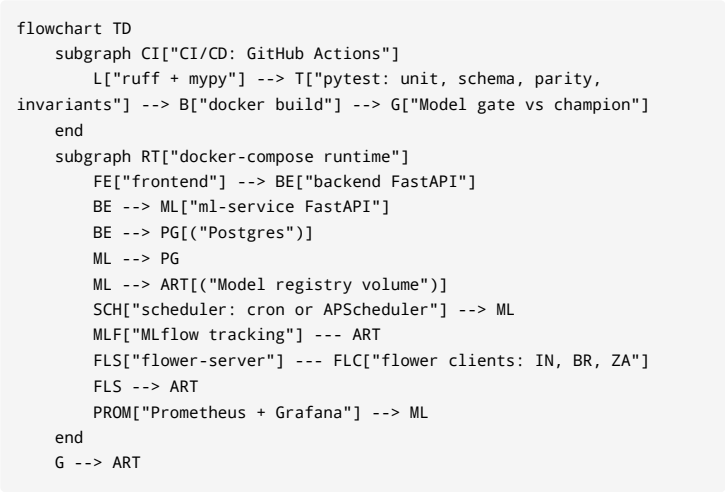
Component	Metrics	Protocol	Acceptance gate (target)
Federation	WAPE per country, worst-country WAPE , rounds to converge	Local vs Central vs Federated (+ personalized)	Federated beats local-only on average and on the weakest country
System	p95 latency, throughput	1,000 PHCs x 15 medicines, 4 vCPU	Full batch under 60 s

Closed-loop headline experiment (90 simulated days, same seeds):
P0 fixed reorder levels only, P1 forecast + safety stock + alerts, P2 P1 + redistribution. Compare stockout events and days, unfilled demand, excess inventory, unnecessary transfers.

Ablations: with vs without censoring mask; with vs without footfall; DemandNet vs LightGBM; Phase 1 vs Phase 2 features; federated vs local-only; safety-stock policy vs fixed rule.

Caveat to state on slides: results are on simulated data; only the pipeline sanity check on a public dataset is not.

17. Deployment flow



Steps

1. CI runs lint, type checks, and tests. Schema and invariant tests block any change that breaks the JSON contract.
2. A candidate model must pass evaluation gates against the current champion.
3. Promotion: **candidate, offline gates, shadow run (3 days, not served), champion**. Rollback is a pointer change to the previous manifest.
4. Runtime is docker-compose: ML service, Flower server and 3 client containers, Postgres, MLflow (file backend is enough), scheduler, monitoring.
5. The ML service is stateless and loads artifacts at start (cached), so it can scale horizontally.

18. APIs (ML service, internal network + API key)

All timestamps are ISO-8601 UTC. Prediction endpoints return a **bare JSON array of exact-contract records**; the model version is returned in the `X-Model-Version` header.

Method and path	Purpose
<code>POST /v1/predict</code>	Predict for listed series at <code>as_of</code>
<code>POST /v1/predict/batch</code>	Run all series for countries at <code>as_of</code> ; returns job id
<code>GET /v1/predictions/latest?country_code=&phc_id=&risk_level=&limit=</code>	Latest stored records (array)
<code>POST /v1/observations</code>	Ingest cycle observations; recompute touched series (MVP-2)
<code>POST /v1/whatif</code>	Recompute one record with a hypothetical stock change; not persisted
<code>POST /v1/alerts/evaluate</code>	Records in, alert candidates out (envelope is a backend concern)
<code>GET /v1/explain/{phc_id}/{medicine_id}?as_of=</code>	Internal meta: quantiles, risk and anomaly components, model version
<code>GET /v1/models</code>	Champion version, metrics, manifest

Method and path	Purpose
GET /v1/health, GET /metrics	Liveness, Prometheus metrics
POST /v1/admin/retrain	Trigger a retrain or FL job (admin key)

Request: predict

```
{
  "as_of": "2026-09-21T08:00:00Z",
  "series": [{"phc_id": "IN_UP_024", "medicine_id": "MED_001"}]
}
```

Response: [{ ...exact record from section 1... }]

Request: what-if (used for "prediction, action, changed system, new prediction")

```
{
  "as_of": "2026-09-21T08:00:00Z",
  "phc_id": "IN_UP_024",
  "medicine_id": "MED_001",
  "stock_delta": 600,
  "applied_at": "2026-09-21T10:00:00Z"
}
```

Response: one exact-contract record recomputed as if the transfer had arrived.

Errors: 422 on schema violation, 409 on stale `as_of` (older than the latest ingested observation), 503 if no champion model is loaded. Records failing the invariants are dropped and logged, never returned.

19. Monitoring

```
flowchart TD
    P["Predictions and meta"] --> J["Join with actuals after horizon"]
    A["Actuals from PHC feed"] --> J
    J --> M1["Rolling WAPE, bias, coverage"]
    J --> M2["Stockout precision, recall, calibration"]
    INF["Incoming features"] --> M3["Drift: PSI, KS vs training"]
    INF --> M4["Data quality: freshness, missing, recon errors"]
    P --> M5["Anomaly rate and alert volume per country"]
    SYS["Service"] --> M6["Latency, errors, batch duration"]
    M1 --> TR1["Thresholds breached?"]
    M2 --> TR1
```

```

M3 --> TR
M4 --> TR
M5 --> TR
M6 --> TR
TR -->|"no"| OK["Continue, weekly report"]
TR -->|"yes"| ACT["Notify owner + open retrain job"]
ACT --> CAND["Candidate model or FL rounds"]
CAND --> SH["Shadow run 3 days"]
SH --> PR{"Beats champion on gates?"}
PR -->|"yes"| PROMO["Promote, keep previous for rollback"]
PR -->|"no"| KEEP["Keep champion"]

```

Monitor	Signal	Alarm (starting values, tune)
Data freshness	Hours since last report per PHC	> 36 h for more than 5% of a country's PHCs
Data quality	Missing rate, reconciliation errors, censored share	Missing > 10% or recon errors > 3%
Input drift	PSI on consumption ratio, footfall, momentum	PSI > 0.25 on at least 3 features
Forecast quality	Rolling 14-day WAPE, bias, coverage once actuals arrive	WAPE worse than the validation baseline by 20% for 7 days; coverage outside 70% to 90%
Anomaly health	Detections per country per day	Above 3x the 30-day median (over-firing) or zero for 14 days (dead)
Risk calibration	Reliability of scores vs realized stockouts	ECE > 0.10
Alert load	Alerts per PHC-week; unresolved P1 age	Volume above cap; P1 unresolved > 6 h
Federation	Per-round validation WAPE per client, dropouts	Client WAPE diverging > 25% from the others
Service	p95 latency, error rate, batch time	Batch > 60 s, error rate > 1%

Retrain triggers: weekly scheduled FL round set; or any forecast-quality or drift alarm; or a new PHC cohort onboarded. Promotion always goes through the shadow and gate flow above.

20. Interfaces, plan, and risks

Team interfaces

With	Contract
Anant (data)	Silver schemas in <code>data/schemas/</code> ; simulator must emit <code>is_censored</code> , <code>event_labels</code> , <code>replenishments</code> ; <code>true_demand</code> in a separate eval-only table
Sharma (redistribution)	The exact JSON only; formulas in section 1; use <code>mock_predictor.py</code> from day 1
Divyansh (backend)	<code>/v1/*</code> endpoints; alert envelope and persistence live in backend; <code>prediction_meta</code> table for internal fields
Frontend	Same JSON; risk level colors from <code>risk.level</code> ; details from <code>/v1/explain</code>

17-day ML plan

Days	Deliverable
1 to 2	<code>output.py</code> contract, mock predictor, baselines, feature spec, data schemas agreed
3 to 5	Features, LightGBM, DemandNet centralized, backtest harness
6 to 7	Stockout ETA, safety stock, risk, JSON assembler, first real end-to-end record
8 to 9	Anomaly layers, surge overlay, event-level evaluation
10 to 12	Federated runs (local vs central vs federated), interval calibration
13 to 14	Backend integration, <code>/whatif</code> , alert rules, monitoring basics
15	MVP-2: 5-hour cycles, velocity overlay, emergency scenario
16	Full evaluation, ablations, closed-loop experiment
17	Freeze, demo scenario, docs

Main risks

Risk	Mitigation
Simulator circularity (model learns the generator)	Independent generator structure, unseen scenario families, public-dataset sanity check, label everything as simulated
Censored demand biases the forecaster low	Censoring flags and the one-sided loss; ablation proves the effect
Federated non-IID performance loss	FedProx, personalization head, report worst-country WAPE
Lead-time data missing or noisy	Catalog default fallback, floor on <code>sigma_L</code> , <code>replenishments</code> table requirement
Alert fatigue	Hysteresis, dedup, priority ranking, alert-volume monitor
Scope creep	Phase 2 features and MVP-2 only after MVP-1 gates pass